

INTRODUCTION TO SOFTWARE ENGINEERING

Exam-Ready Study Notes

Lectures 01–10

Primary source: uploaded course lecture slides

Supplementary grounding: established terminology and principles from Ghezzi et al., Hamlet & Maybee, Pressman, and standard software engineering practice.

Purpose

A single revision document capturing the concepts, terminology, models, examples, comparisons, and examinable principles presented across the supplied lectures. Repeated introductory slides are consolidated without losing the concepts they introduce.

Important source note: Numerical industry statistics and slide-specific examples are retained as lecture content rather than independently re-verified. Where textbook knowledge is used to clarify a concept, it is presented as standard software engineering terminology rather than as a claim about what the slides explicitly stated.

Table of Contents

Lecture sequence and major revision topics.

Section	Topic
1	Lecture 01 — Introduction to Software Engineering
2	Lecture 02 — Introduction (Reinforcement)
3	Lecture 03 — Nature of Software and Software Evolution
4	Lecture 04 — Software Quality
5	Lectures 05–06 — Quality, Domains, Process, Lifecycle
6	Lectures 07–08 — Software Engineering Principles
7	Lecture 09 — Change, Generality, Incrementality, Abstraction, Reuse
8	Lecture 10 — SE and Other Disciplines
9	Integrated Exam Revision — High-Value Comparisons
10	Rapid Revision Sheet

How to Use These Notes

These notes follow the lecture sequence from Introduction through software engineering principles and the relationship of software engineering to other disciplines. Because Lectures 01–02 and parts of Lecture 03 repeat material, repeated statements are consolidated into one coherent explanation while retaining every substantive concept.

Exam strategy

- **Definitions:** memorize the formal meaning first; then learn one distinguishing characteristic and one example.
- **Comparisons:** answer using the same dimensions on both sides (purpose, dependencies, quality focus, lifecycle behavior, or stakeholder perspective).
- **Models:** write the stages in order and state the purpose of each stage.
- **Principles:** state the principle, explain the design consequence, and give a small example.
- **Quality:** distinguish correctness, reliability, robustness, performance, efficiency, usability, verifiability, maintainability, repairability, evolvability, portability, interoperability, and security.

Source coverage

Lecture	Primary coverage in these notes
01	Introduction, software, engineering, SE definition, team, lifecycle, quality, defects, costs, challenges
02	Reinforcement of Lecture 01 concepts; consolidated to avoid redundant prose
03	Fault/failure, nature of software, evolution, Lehman's laws, transformation examples
04	Software quality, product/process, internal/external characteristics, core quality attributes
05–06	Quality attributes, quality models, application domains, legacy software, process, lifecycle, umbrella activities
07–08	Rigor/formality, separation of concerns, modularity, coupling, cohesion, modularization trade-offs
09	Anticipation of change, OCP, generality, incrementality, abstraction, customers, terminology, documentation, transformation, reuse
10	Compiler case study and relationships among SE, CS, management science, systems engineering, and other disciplines

1. Lecture 01 — Introduction to Software Engineering

1.1 Software Engineering: the central idea

The lectures frame software engineering around **problem solving, computing, and multi-person construction of multi-version software**. The central definition is a disciplined, consistent, and systematic effort to construct (design and build) and maintain software in a timely and cost-effective manner. This definition emphasizes that software engineering is broader than programming: it includes technical activities, people, process, management, quality, and evolution.

1.2 What is software?

Software is not merely source code. In the lecture definition, it consists of **computer programs together with associated configuration data and documentation** needed for correct operation. Configuration data helps establish the required operating setup; system documentation explains system structure; and user documentation explains how the system is used.

1.3 What does engineering mean?

- Make things work under real constraints.
- Apply theories, methodologies, and tools appropriately.
- Provide solutions when no directly applicable theory or method exists.
- Recognize financial and organizational constraints, not only technical feasibility.

Therefore, software engineering is the application of engineering thinking to software production: solve computing-related problems systematically while remaining concerned with all significant aspects of production.

1.4 Aspects of software production

- Size of the software system.
- Complexity of the system.
- Need for and involvement of teams.
- Technical development process.
- Management of projects and teams.
- Development and use of tools, theories, and methods that support software production.

1.5 Why software engineering is needed

Software is embedded in everyday products and critical systems: appliances, routers, temperature-control systems, surgical equipment, search engines, chatbots, aircraft, and defense systems. The lecture connects software engineering practice with producing good software economically and with improving living standards.

A useful exam formulation is: **SE is needed because software systems are pervasive, increasingly complex, developed by teams, maintained over long lifetimes, and constrained by quality, cost, schedule, and operational requirements.**

Key Points to Remember

- Software includes programs plus configuration data and documentation.
- Software engineering is broader than coding; it covers the full production and maintenance context.
- Engineering explicitly recognizes technical, financial, organizational, and operational constraints.
- The lecture definition emphasizes disciplined, consistent, systematic, timely, and cost-effective work.

1.6 Software engineering team and stakeholders

Software development is a multi-person activity. Stakeholders include people who commission, specify, build, test, deploy, operate, maintain, document, or otherwise depend on the software.

Role	Primary responsibility
Requirements analyst	Works with customers to identify and document requirements.
Designer	Produces a system-level description of what the system is supposed to do and how it is structured.
Programmer	Implements the design in source code.
Tester	Finds faults and provides evidence about software behavior.
Trainer	Shows users how to operate the system.
Maintenance team	Corrects faults and performs changes after delivery.
Librarian / documentation role	Prepares and stores documents such as requirements.
Configuration management team	Maintains correspondence and consistency among software artefacts and versions.

1.7 Software lifecycle

The lecture presents a lifecycle as an ordered set of major activities used to develop and deliver a software product. The core phases are:

- 1 **Requirements analysis and definition:** determine what the system must provide and under what constraints.
- 2 **System/architecture design:** establish the high-level structure and major components.
- 3 **Program/detailed design:** specify component-level algorithms, interfaces, and procedures.
- 4 **Coding/implementation:** translate design into executable software.
- 5 **Testing:** unit, integration, and system testing provide progressively broader verification.
- 6 **Deployment:** deliver/install the system for operational use.
- 7 **Maintenance:** modify the software after delivery to correct defects, adapt to environments, and add or improve functionality.

Textbook clarification: lifecycle models such as waterfall, incremental, iterative, spiral, and agile approaches differ mainly in *how these activities are organized, repeated, and controlled*; the existence of the underlying engineering activities is not eliminated.

1.8 Software engineering and software quality

The lecture stresses that software engineers should adopt systematic and organized approaches to produce high-quality software. However, real engineering may also require pragmatic or ad hoc techniques when an elegant theory from computer science does not directly solve the practical problem. This does not mean abandoning engineering discipline; it means combining established methods with informed problem solving.

Key Points to Remember

- Know the responsibilities of the major SE roles.
- Know the lifecycle phases in order.
- Testing is not one single activity: the slides explicitly distinguish unit, integration, and system testing.
- Maintenance is a lifecycle activity, not an optional afterthought.

2. Lecture 02 — Introduction (Reinforcement)

Lecture 02 repeats the central introductory concepts from Lecture 01: software, engineering, software engineering, software production aspects, the development team, lifecycle, software quality, defects, costs, and engineering challenges. The repeated material is consolidated here into the examinable relationships rather than copied verbatim.

2.1 From problem solving to software engineering

The lecture progression can be written as: **problem** → **computing/computer-based solution** → **systematic engineering approach** → **team-based construction** → **multiple versions and maintenance**. The important conceptual shift is from merely making a program execute to managing the entire process by which a useful software product is produced and evolved.

2.2 Defects, quality, cost, and success

The lecture connects communication failures, faulty requirements, and documentation/design/coding errors with defects. Defects can affect cost and schedule, customer satisfaction, and overall project success. The lecture states the practical relationship that reducing defects tends to increase software quality.

The slides also present a defect-correction cost curve in which the relative cost of fixing a defect increases substantially when the defect is discovered later. The lecture values shown are **1, 2.5, 6.5, 16, 40, and 110** across progressively later stages. These values should be treated as lecture-specific illustrative ratios, not universal constants.

Concept	Exam-ready interpretation
Defect	A failure to meet stated requirements or user/customer expectations.
Quality	A multi-faceted property; examples include fitness for use, conformance to requirements, freedom from deficiencies, and lack of defects.
Early defect prevention	Finding/preventing defects earlier generally reduces downstream rework and correction cost.
Project success	Affected by requirements quality, communication, technical work, schedule, cost, and customer satisfaction.

2.3 Engineering conflicts

Software projects must balance acceptable quality with usability, security, reliability, performance, cost-effectiveness, engineering feasibility, operational feasibility, a limited budget, and timely completion under limited time. These concerns can conflict; engineering therefore involves managing trade-offs rather than maximizing every attribute independently.

2.4 Algorithmic examples in the lecture

The slides use small C++ examples to illustrate that software engineering includes examining algorithms and code, not merely writing syntax. Examples involve counting frequencies and finding a second-largest value. The class exercise asks for sorting ten unique non-negative integers in the range 0–20 without nested loops, allowing additional memory. The intended engineering lesson is to recognize that a problem may admit a different design when constraints change.

A standard solution idea for the sorting exercise is **frequency/counting-based organization**: because the value range is small and known, an auxiliary array indexed by value can be used, followed by a linear scan to emit values in ascending order. This is an example of selecting an algorithm based on the problem's constraints rather than defaulting to a general comparison sort.

Key Points to Remember

- Defects arise from problems in requirements, communication, design, coding, and documentation.
- The later a defect is discovered, the more expensive correction tends to be.
- Quality is multi-dimensional and must be considered together with cost and schedule.
- Algorithm choice should reflect the input constraints and required performance.

3. Lecture 03 — Nature of Software and Software Evolution

3.1 Fault, error, and failure

The lecture distinguishes a **fault** from a **failure**. A fault occurs when a human makes a mistake (an error) while performing a software activity and that mistake becomes incorporated into a software work product. A failure is a departure of the running system from its required behavior.

Term	Meaning	Simple example
Error	Human mistake in an activity.	Developer misunderstands a requirement.
Fault (defect)	A flaw introduced into a software artefact as a result of an error.	Condition uses < instead of <=.
Failure	Observed departure from required behavior during execution.	Program rejects a valid boundary input.

The distinction is important because not every fault immediately produces a failure: the faulty code may execute only under a particular input or state. Testing attempts to expose failures, while reviews, analysis, and other verification activities can detect faults before execution.

3.2 Nature of software

- **Malleable:** software can be changed relatively easily compared with physical artefacts, but frequent change can create complexity and maintenance burden.
- **Human-intensive:** software creation depends heavily on human reasoning, communication, design, coding, review, and coordination.
- **No physical wear-out in the hardware sense:** software does not physically deteriorate through use like a mechanical component. Failures generally arise from latent faults, changes, interactions, or environmental conditions.
- **Versioned and evolutionary:** software commonly exists in successive releases and must be modified as requirements, platforms, and operational environments change.

3.3 Hardware versus software failure behavior

The slides contrast hardware and software failure curves. Hardware may exhibit physical wear-out; software does not exhibit the same intrinsic wear-out mechanism. Software reliability can nevertheless decline operationally when modifications introduce defects or when the environment changes and the software is not adapted.

3.4 Software evolution

Software evolution is the continuing modification of a software system after its initial delivery. Changes may be corrective (fixing defects), adaptive (responding to environmental/platform changes), or perfective (improving or extending functionality). Evolution creates a need for maintainable architecture, controlled change, regression testing, documentation, and configuration management.

3.5 Lehman's laws presented in the lecture

Law	Lecture meaning
Continuing Change (1974)	A system must be continually adapted or it becomes progressively less satisfactory; eventually replacement or restructuring may become more economical.
Increasing Complexity / Entropy (1974)	System complexity tends to increase with time unless work is done to maintain or reduce it.
Continuing Growth (1991)	Functional content must continue to increase to maintain user satisfaction over the system's lifetime.
Declining Quality (1996)	Perceived quality tends to decline unless the system is rigorously maintained and adapted to operational-environment changes.

Key Points to Remember

- Error is the human mistake; fault/defect is the resulting flaw; failure is the observed incorrect behavior.
- Software is malleable and versioned, but change can increase complexity.

- Software does not physically wear out in the same way as hardware.
- Lehman's laws explain why long-lived software requires continuous evolution and maintenance.

3.6 Time-series and transformation examples

The lecture introduces examples in which software requirements are expressed as transformations or services. One example involves farmers, gardens, and fruit-picking assignments. Another states that a time-series model is needed to forecast demand. A related requirement says the forecast output should be written to a file and that the software should be runnable once for multiple businesses.

These examples emphasize a recurring software-engineering skill: turn an informal problem statement into explicit inputs, outputs, constraints, assumptions, and reusable operations. For the forecasting example, an exam answer should identify the distinction between the *core computation* (forecasting) and surrounding concerns such as input data, output persistence, repeated execution, and support for multiple business contexts.

3.7 Software as a transformation

A useful abstract view is to model software as a transformation from a set of inputs to a set of outputs. The engineering task is not merely to write the transformation; it is to characterize the domain of valid behavior.

- 1 Identify all relevant inputs and outputs.
- 2 Identify valid and invalid input/output ranges.
- 3 Identify special or significant input/output values.
- 4 Identify important combinations of input and output values.
- 5 Define the transformation that maps valid inputs to required outputs.
- 6 Consider how invalid or exceptional inputs should be handled.

This viewpoint directly supports requirements analysis and testing. Once valid/invalid ranges and significant combinations are known, they can become test conditions and acceptance criteria.

3.8 Example: online telephone directory

The lecture's online directory example requires a service that, when given a person's name, provides a phone number; it also lists area codes for different parts of the country and can generate emergency telephone numbers for the user's area on request. From an engineering perspective, the example contains multiple operations that share a domain and can be specified through explicit inputs, outputs, preconditions, and exceptional cases.

Operation	Possible input	Expected output
Lookup	Name	Phone number or defined 'not found' result
Area-code information	Geographic region	Associated area-code information
Emergency service	User's area	Emergency telephone numbers

3.9 Connection to testing

The transformation model provides a bridge from requirements to tests: test normal values, boundary values, invalid values, special values, and important combinations. This is a standard basis for black-box test design and complements structural testing of the implementation.

Key Points to Remember

- Transformation thinking forces explicit identification of inputs, outputs, domains, and exceptional conditions.
- A requirement may contain multiple related transformations/services.
- The transformation view naturally supports test-case design.
- The forecasting example illustrates separating core computation from reusable operational context.

4. Lecture 04 — Software Quality

4.1 What is a good software product?

The lecture emphasizes that good software engineering must include a strategy for producing quality software. Quality is not a single scalar property; it has multiple facets and is perceived differently by different stakeholders.

Users commonly judge **external characteristics**: whether required functions work, how often failures occur, and how failures affect them. Designers and maintainers also judge **internal characteristics**, such as structural organization and ease of modification. Consequently, two stakeholders can evaluate the same product using different quality criteria.

Perspective	Typical focus
User / customer	Visible behavior, correct functionality, failures, usability, response time, security.
Developer / designer	Architecture, modularity, understandability, verifiability, maintainability.
Maintainer	Ease and cost of correcting, adapting, and evolving the system.

4.2 Product and process

Product is the software delivered to the customer. **Process** is the software production process. Work products such as requirements documents, design models, source code, test artefacts, and configuration records are outputs of process activities even when they are not the final executable product.

4.3 Core software quality attributes

Quality attribute	Exam-ready definition
Correctness	Degree to which the software behavior is equivalent to its specification; the lecture describes this as a mathematical property.
Reliability / dependability	Probability that the software operates as expected over a specified time interval under stated conditions.
Robustness	Ability to behave reasonably when confronted with unexpected inputs or situations.
Performance	Amount of useful work performed per unit time, often reflected through response time, throughput, or latency.
Efficiency	How economically the software uses computer resources such as CPU time, memory, storage, or network capacity.
Usability	How easily and effectively human users can learn and use the system.
Verifiability	Ease with which relevant properties of the software can be verified or checked.
Maintainability	Ease and economy with which the software can be modified over its lifetime.
Repairability	Ability to correct defects in a reasonable time and with reasonable effort.
Evolvability	Ability to change the system economically to satisfy new or changed requirements.

Key Points to Remember

- Quality is multi-dimensional and stakeholder-dependent.
- Correctness is relative to a specification; reliability is probabilistic and time-related.
- Robustness concerns unexpected conditions; usability concerns human interaction.
- Maintainability concerns change; repairability concerns defect correction; evolvability concerns new/changed requirements.

5. Lectures 05–06 — Quality Attributes, Models, Application Domains, and Process

5.1 Additional quality attributes

Attribute	Meaning and significance
Reusability	Extent to which software components or knowledge can be reused in other systems or contexts.
Portability	Ability to run in different environments with limited modification.
Understandability	Ease with which developers can understand the software's behavior and structure relative to its complexity.
Interoperability	Ability to coexist and cooperate with other systems through compatible interfaces and interactions.
Security	Ability to protect information and resist unauthorized access or misuse; the lecture connects it with low vulnerability and with confidentiality/integrity.
Productivity	Amount of work performed by a software team in a unit of time; also connected in the slides with process efficiency.
Timeliness	Ability to deliver a product when required.
Visibility	Degree to which process steps and their current status are documented and observable.

5.2 Understandability and control structure

The lecture contrasts code in which control flow jumps between labels with a structured conditional form. The structured form is easier to follow because the reader can reason about the conditions locally instead of tracing non-local jumps. The example illustrates that restructuring code can improve understandability without changing the intended decision logic.

```
// Structured form shown conceptually in the lecture
if (age < 55)    benefit = minimum;
elseif (age < 65) benefit = minimum + bonus;
elseif (age < 75) benefit = minimum * 1.5 + bonus;
else          benefit = maximum;
```

The engineering lesson is broader than avoiding one specific language feature: **control structure should support human comprehension, local reasoning, testing, and maintenance.**

5.3 Quality models

The slides introduce several quality-modeling approaches. A quality model provides an organized vocabulary for discussing and assessing software quality; it does not imply that one metric can fully represent quality.

Model	Lecture coverage / exam focus
McCall's Quality Model	Groups product qualities into operation characteristics, ability to undergo change, and adaptability to new environments. The slide points to Pressman, Chapter 15 (9th ed.).
FURPS	Functionality, Usability, Reliability, Performance, Supportability. Supportability includes concerns such as installation and problem localization; maintainability is closely related.
ISO/IEC 25010	The lecture distinguishes Quality in Use —characteristics relevant when using software in a particular context—from Product Quality —characteristics of the software system's static and dynamic properties. The slides identify versions in 2011 and 2017.

Textbook clarification: ISO/IEC 25010 is commonly used as a structured quality vocabulary. In exams, the safest approach is to reproduce the model categories taught in class and distinguish quality-in-use from product-quality perspectives.

5.4 Software application domains

Domain	Description / examples
System software	Services other programs; examples include compilers, editors, and operating-system components.
Application software	Stand-alone software solving a specific business/user need, such as leave-management or point-of-sale systems.
Engineering/scientific software	Simulates or computes scientific/engineering phenomena such as astronomy, volcanology, or molecular biology.
Real-time systems	Must respond to events within predefined and strict time limits; response timing is part of correctness.
Embedded systems	Resides within a product/system and performs dedicated functions; examples include temperature-control systems and microwave keypads.
Product-line software	Software family for a group of related products marketed under a common brand; the slide uses MS Office as an example.
Web/mobile applications	Browser-based or mobile-device-based software.
Artificial-intelligence software	Uses heuristic or AI techniques for complex problems, such as robotics, expert diagnosis, or forecasting.

5.5 Legacy software

Legacy software is old software that continues to perform important core functions and is therefore often indispensable to a business. The difficulty is that it may have been repeatedly modified over many years without corresponding architectural restructuring.

- Continually modified to accommodate changing requirements and platforms.
- Difficult and costly to change further.
- Inextensible designs and convoluted code.
- Poor or nonexistent documentation.
- Unarchived test results and poorly managed change history.
- Risky to evolve despite continued business importance.

Legacy modernization is therefore not simply a coding exercise. Engineers may need to recover requirements, understand architecture, establish regression tests, improve documentation, and introduce controlled interfaces before large-scale changes become safe.

5.6 Application domains and relevant quality

Domain	System characteristics	Highly relevant quality concerns
Information systems	Data-oriented; storage/retrieval; banking, library, HR systems.	Data integrity, security, availability, transaction performance.
Real-time systems	Event/control-oriented; sensors; strict response deadlines.	Correctness including timing, reliability, safety.
Distributed systems	Multiple computers connected by a communication network.	Portability, interoperability, performance, reliability.
Embedded systems	Hardware-specific; interfaces with other system components.	Efficiency, robustness; often reliability and safety are also critical.

Human-computer quality concerns such as usability can be relevant across all these domains.

5.7 Selecting quality attributes for a problem

The class activities train the student to choose relevant qualities and explain how to measure them. For an algorithm that sorts unique non-negative integers, correctness can be checked by verifying ascending order and preservation of all inputs; efficiency can be measured through time/space complexity or empirical runtime; robustness can be evaluated against invalid or unexpected inputs if such inputs are within the system's responsibility.

For a point-of-sale system, correctness, security, reliability, usability, performance, efficiency, maintainability, and verifiability can all matter. The measurement method should match the attribute: for example, response-time percentiles for performance, failure frequency/MTBF-type measures for reliability, defect rates and review/test coverage for verifiability, and task success/time/error rates for usability.

Key Points to Remember

- Quality attributes must be selected in relation to the application domain.
- Real-time correctness includes timing constraints, not merely numerical output.
- Legacy systems are difficult to evolve but may be business-critical.
- A quality attribute is not fully understood until an appropriate way of assessing it is identified.

5.8 Software engineering process

A **process** is a series of steps involving activities, constraints, and resources to produce an intended output. Processes impose structure on work and can be represented with tools, techniques, diagrams, and notations.

Why is a process useful?

- Imposes consistency and structure on activities.
- Guides understanding, control, examination, and improvement of activities.
- Allows experience to be captured and transferred to others.

5.9 Framework activities

Framework activity	Purpose
Communication	Interact with customers and other stakeholders to establish a shared understanding.
Planning	Create a roadmap, estimates, schedule, resources, and project plan.
Modeling	Understand requirements and provide analysis/design representations.
Construction	Generate code and perform testing.
Deployment	Deliver the software, obtain feedback, and evaluate operational use.

A common conceptual flow is: **Communication** → **Planning** → **Modeling** → **Construction** → **Deployment**, with feedback and iteration possible depending on the lifecycle model.

5.10 Waterfall and software lifecycle

The lecture names the waterfall model and separately gives the lifecycle activities of requirements, architecture design, detailed design, implementation, testing, deployment, and maintenance. The classical waterfall idea organizes development into largely sequential phases with formal handoffs. Its main exam characteristic is the emphasis on ordered progression and relatively stable baselines.

Textbook clarification: practical projects rarely have perfectly one-directional development. Requirements, design, implementation, and testing often feed information back to earlier activities. The important exam distinction is the *modeling of the process* versus the fact that real engineering requires feedback and change control.

5.11 Umbrella activities

Umbrella activity	Purpose
Software Quality Assurance (SQA)	Planned activities that provide confidence that processes and products meet relevant quality requirements.
Configuration Management (CM)	Controls versions, baselines, changes, and relationships among software artefacts.
Technical Reviews (TR)	Systematic examination of work products to detect defects and improve quality early.
Project Tracking and Control (PTC)	Compare actual progress with plans and take corrective action.
Risk Management (RM)	Identify, analyze, monitor, and respond to project/product risks.

5.12 Software engineering practice

- 1 **Understand the problem** — communication and analysis.
- 2 **Plan a solution** — modeling and design.
- 3 **Carry out the plan** — code generation/implementation.
- 4 **Examine the results for accuracy** — testing and quality assurance.

Key Points to Remember

- A process gives structure, repeatability, control, and opportunities for improvement.
- Remember the five framework activities and their purposes.
- Remember the seven major lifecycle activities shown in the lecture.
- Umbrella activities span the technical lifecycle rather than replacing it.

- The practice cycle is understand → plan → execute → examine.

6. Lectures 07–08 — Software Engineering Principles

6.1 Rigor and formality

Rigor means working with precision and exactness so that reliable products can be produced repeatedly, costs can be controlled, and confidence in reliability can be increased. **Formality** is the highest degree of rigor and uses mathematical laws or formal reasoning to derive and evaluate aspects of the software process and product.

The lecture associates rigor/formality with benefits for reliability, verifiability, maintainability, reusability, portability, understandability, and interoperability. Formality is especially valuable when ambiguity or failure has high cost, although the appropriate degree of rigor depends on the problem and its risks.

6.2 Separation of concerns

Separation of concerns means isolating different aspects of a problem or system so that each can be reasoned about with less interference from unrelated aspects. The lecture lists several ways to separate concerns:

- By **time** — distinguish activities or concerns occurring at different lifecycle times.
- By **qualities** — isolate concerns such as performance, security, or usability.
- By **views** — examine the same system through different representations.
- By **parts** — divide the system into components/modules.
- By **management vs technical** — separate project-management concerns from engineering tasks.
- By **problem domain vs implementation domain** — distinguish what the system must accomplish from how it is implemented.

6.3 Modularity

Modularity is the principle of keeping unrelated aspects of a system separate so that each aspect can be studied in isolation. A well-modularized system tends to contain modules with a clear purpose and relatively limited dependence on other modules.

- **Decomposability**: the system can be broken into manageable parts.
- **Composability**: parts can be combined in useful ways.
- **Fault localization**: fewer modules are plausible suspects when a fault occurs.
- **Change isolation**: changes to one module are less likely to force widespread changes elsewhere.
- **Understandability**: each module can be understood with less global context.

The lecture connects module independence to **coupling** and **cohesion**, and also invokes the single-responsibility idea: a module should have a focused purpose rather than unrelated responsibilities.

6.4 Coupling

Coupling measures how strongly modules depend on one another. **Tightly coupled** modules have many or strong dependencies; **loosely coupled** modules have weaker interconnections; **uncoupled** modules have no direct interconnections.

Slide terminology	Meaning / exam cue
Content coupling	One component depends on or modifies the internal workings/data of another, or branches into its internal control flow. Very strong dependence.
Common coupling	Components depend on common/shared data; changes require tracing all users of that data.
Control coupling	One component passes information that controls the activity/flow of another component.
Routine-call coupling	One module invokes another through a procedure/function call.
Data coupling	Modules communicate by passing data values through a relatively simple interface.
Stamp coupling	A complex data structure is passed; both sides depend on its format/organization.
Type-use coupling	A module depends on a type defined by another module.

Slide terminology	Meaning / exam cue
Import coupling	A module depends on imported declarations/types/services from another module/package.

Textbook note: coupling taxonomies differ across authors and languages. The slide's list should be memorized as the course vocabulary; the general engineering principle is that unnecessary coupling should be reduced because it increases change impact and complicates reasoning.

6.5 Coupling examples and consequences

```
// Control coupling: the argument controls the callee's behavior
bool act(int x) {
    if (x == 0) return false;
    else return true;
}

void call() {
    act(1);
}

// Stamp/type-use example from the lecture
class A { /* ... */ };
class B {
    A var;                // type-use dependency
    void calculate(A data) { // structured object passed
        /* ... */
    }
};
```

High coupling is undesirable when it is unnecessary because a change in one module can propagate into many other modules. Shared data and direct internal access are particularly difficult to reason about because the source of a value or dependency may be far from the point where its effect is observed.

6.6 Cohesion

Cohesion measures the degree to which the internal elements of a module are related to one another and collectively contribute to the module's assigned task. High cohesion is desired: a cohesive module has a focused purpose and its data/functions naturally belong together.

The lecture slide names 'types of cohesion' without listing them. Standard textbook terminology commonly orders cohesion from weaker to stronger as follows:

Type	Exam-ready description
Coincidental	Elements have little meaningful relationship and are grouped arbitrarily.
Logical	Elements perform logically similar operations selected by a control flag or mode.
Temporal	Elements are grouped because they occur at the same time, such as initialization.
Procedural	Elements are related by a sequence of control steps but do not necessarily share one focused data purpose.
Communicational	Elements operate on the same data set or contribute to the same input/output context.
Sequential	Output from one part becomes input to the next.
Functional	All elements contribute to one well-defined function; strongest classic form.

The key relationship is: **high cohesion within modules + low unnecessary coupling between modules → stronger modular independence.**

6.7 Over-modularization versus under-modularization

Modularity is not maximized by creating the largest possible number of modules. **Under-modularization** produces large, difficult-to-understand components with mixed responsibilities. **Over-modularization** creates excessive interfaces, indirection, coordination cost, and fragmentation. Good design seeks modules that are understandable, cohesive, and appropriately independent.

6.8 Intellectual control and the 7 ± 2 rule

The lecture revisits separation of concerns and modularity as mechanisms for maintaining **intellectual control**: engineers should be able to understand and reason about independent parts rather than hold the entire system in working memory at once. The slide mentions the 7 ± 2 rule as a human-comprehension heuristic. It should be treated as a heuristic rather than a precise law of software design.

Key Points to Remember

- Rigor improves precision and confidence; formality is rigor using formal/mathematical reasoning.
- Separation of concerns isolates unrelated dimensions of a system.
- Modularity aims for manageable, focused, relatively independent parts.

- High cohesion is desired; unnecessary high coupling is undesirable.
- Know the course coupling terms and the classic cohesion types.
- Avoid both under-modularization and over-modularization.

7. Lecture 09 — Principles for Change, Generality, Abstraction, Documentation, and Reuse

7.1 Anticipation of change

Software systems operate in changing environments. **Evolvability** is therefore a design objective: the system should be changeable without disproportionate cost or risk. **Reusability** similarly encourages components and specifications that can serve more than one context.

7.2 Open–Closed Principle (OCP)

The standard formulation of the Open–Closed Principle is: **software entities should be open for extension but closed for modification**. In practical terms, new behavior should often be introduced through new implementations, configurations, or extensions rather than repeatedly editing stable, widely used core logic.

The lecture contrasts a design made without anticipating change with one designed with anticipated variation. The engineering lesson is not 'never modify code'; rather, it is to identify likely variation points and place abstractions/interfaces around them when doing so reduces future change cost.

7.3 Generality

Generality means avoiding unnecessary commitment to a single context when a more general interface can serve a useful family of contexts. The lecture presents procedure interfaces in increasing generality:

```
PROCEDURE SUM: INTEGER
POSTCONDITION: returns sum of 3 global variables
```

```
PROCEDURE SUM(a, b, c: INTEGER): INTEGER
POSTCONDITION: returns sum of parameters
```

```
PROCEDURE SUM(a[]: INTEGER): INTEGER
POSTCONDITION: returns sum of elements in array a
```

The progression parameterizes context-specific information: global variables → explicit parameters → an array representing an arbitrary number of values. Generality can improve reuse, but it has trade-offs: a more general interface may be harder to understand, harder to validate, less specialized for performance, or more complex than the immediate problem requires.

7.4 Incrementality

Incrementality means developing and delivering software in **increments** rather than waiting for the complete system before obtaining useful feedback. Early subsets and prototypes allow stakeholders to observe behavior and reveal misunderstandings earlier.

- 1 Select a coherent subset of functionality.
- 2 Design and implement that subset.
- 3 Evaluate it with users/stakeholders or through testing.
- 4 Use the feedback to refine requirements/design and plan the next increment.
- 5 Repeat until the required system is completed.

Incrementality reduces the risk of discovering fundamental misunderstandings only at the end of the project. It also supports prioritization and earlier delivery of useful functionality.

7.5 Abstraction

An **abstraction** is a model or representation that omits some details so that attention can be focused on other details. The lecture demonstrates abstraction using sorting.

Level	Sorting description
High-level requirement	Sort list L in non-decreasing order.
Algorithm level	Repeatedly find the smallest remaining element and interchange it with the current position.
Detailed algorithm	Specify loop bounds, indices, comparisons, temporary storage, and exact assignments.

Level	Sorting description
Low-level implementation	Represent the behavior using language-specific or machine-level instructions.

The essential idea is **progressive refinement**: each lower level exposes implementation detail hidden by the higher-level abstraction.

7.6 Abstraction at multiple levels

The lecture also shows a progression from assembly-style source to binary representation. The exact syntax shown is a teaching example; the conceptual point is that a single computational intent can be represented at several levels of abstraction.

```
LDF R2, y          ; load y
ADDF R2, R2, #5.0   ; add 5.0
STF x, R2           ; store result in x
```

At a higher level, the intent might simply be $x = y + 5.0$. At a lower level, register allocation, instructions, addresses, calling conventions, and machine encoding become explicit. The more detail that is exposed, the less abstract the representation becomes.

7.7 Identification of 'customers'

The lecture uses the term **customer** broadly to mean a consumer of another activity's output. This is useful for understanding dependencies between engineering stages:

- A programmer is a customer of the designer.
- A designer is a customer of the requirements engineer.
- A test engineer is a customer of the outputs of other phases.
- A programmer is also a customer of test-engineering information.
- A user is a customer of the maintenance phase.

The point is that every phase produces work products that another stakeholder or activity depends upon. Quality therefore cannot be isolated to one phase.

7.8 Consistent terminology

The lecture asks questions such as 'Specification or Design?' and 'Test has failed?' to highlight ambiguity caused by inconsistent terminology. Software engineering benefits from a shared vocabulary in which terms are defined once, used consistently, and distinguished according to their role in the lifecycle.

7.9 Documentation

- **Record lest you forget it:** documentation preserves decisions and knowledge.
- **Document assumptions:** make constraints and reasoning explicit at each stage.
- **Traceability:** connect requirements to design elements, code, tests, and changes.
- **Document structure:** place information where readers expect it and organize it predictably.
- **Clarity:** write so that intended meaning is not dependent on the author's memory.
- **Terminology:** define important terms and use them consistently rather than repeatedly introducing competing meanings.
- **Indexing:** facilitate retrieval of information in large documentation sets.

7.10 Reuse

Reuse involves knowing existing libraries/components and developing new software so that it can itself be reused. Reuse can occur at the requirements, design, architecture, library, component, and code levels.

A key engineering technique is to define well-specified interfaces (for example, function prototypes or API contracts). A clear interface separates what a component promises from how it implements that promise, enabling independent use and replacement.

Key Points to Remember

- OCP: open for extension, closed for modification.
- Generality removes unnecessary context-specific assumptions but introduces trade-offs.
- Incrementality provides early subsets and feedback.
- Abstraction hides detail; refinement progressively exposes implementation detail.

- Engineering phases are connected through customer/provider relationships.
- Consistent terminology, documentation, traceability, and reusable interfaces reduce coordination and maintenance cost.

8. Lecture 10 — Software Engineering and Other Disciplines

8.1 Case study: software engineering principles in a compiler

A compiler is a useful case study because it is a substantial software system containing multiple well-defined transformations. The lecture lists the major compiler components below.

Component	Function
Lexical analyzer (scanner)	Reads source characters and groups them into meaningful tokens such as keywords, identifiers, and operators.
Syntax analyzer (parser)	Arranges tokens into a tree structure such as a parse tree/abstract syntax tree and checks compliance with grammar rules.
Semantic analyzer	Checks meaning-related constraints such as type compatibility and use of declared identifiers.
Intermediate code generator	Produces a machine-independent intermediate representation that bridges high-level source and low-level target code.
Code optimizer	Improves the intermediate representation so the final program can execute faster and/or use fewer resources.
Code generator	Translates optimized intermediate code into a target language, commonly assembly or machine code.

8.2 Compiler pipeline as a transformation

- 1 Source characters are transformed into tokens.
- 2 Tokens are transformed into a syntactic structure.
- 3 The structure is checked for semantic validity.
- 4 A machine-independent intermediate representation is generated.
- 5 The representation is optimized.
- 6 The optimized form is translated to target code.

This pipeline demonstrates **separation of concerns**: lexical, syntactic, semantic, optimization, and code-generation concerns are assigned to distinct stages. It also demonstrates **modularity**, because each stage has a focused responsibility and communicates through defined representations.

8.3 Quality principles visible in the compiler case

- **Correctness**: generated code must preserve the required meaning of the source program.
- **Reliability**: compiler behavior should be predictable across supported inputs and environments.
- **Performance/efficiency**: optimization can improve generated program performance and resource use.
- **Verifiability**: separate stages permit targeted tests and checks.
- **Maintainability**: modular compiler phases allow one concern to evolve with reduced impact on others.
- **Portability**: an intermediate representation can help separate source-language concerns from target-machine details.
- **Understandability**: clear stage boundaries make the architecture easier to reason about.

The slide includes an assembly-code output example and warns that LLM-generated code should be used with caution. In an exam, the important point is the compiler architecture and the purpose of each phase, not memorization of the exact instruction syntax.

8.4 Software engineering and computer science

Computer science (CS) develops theories, algorithms, abstractions, languages, and computational principles. Software engineering uses such knowledge while addressing the broader problem of building, evolving, and managing software systems under real constraints.

CS area in the lecture	Connection to software engineering
Programming languages	Languages, semantics, compilers, and language mechanisms support implementation and tooling.
Modularity	CS concepts of abstraction and decomposition inform SE architecture and design.
Exception handling	Language/runtime mechanisms support robust system behavior.
Operating systems	Provide foundational services on which engineered applications depend.
Databases	Provide storage, retrieval, transactions, and data-management capabilities for large systems.
Artificial intelligence	Provides algorithms and techniques used in intelligent software.
Theoretical CS	Provides formal foundations, computational models, complexity analysis, and reasoning tools.

8.5 Software engineering and management science

Software engineering is a multi-person endeavor, so technical design must be coordinated with management activities. The lecture identifies:

- Project estimation.
- Project scheduling.
- Human resource planning.
- Task decomposition.
- Task assignment.
- Project tracking.
- Hiring, motivation, and management of people.

The engineering implication is that a technically correct design can still fail as a project if resources, schedule, communication, or coordination are unmanaged. Management activities therefore complement, rather than replace, technical engineering.

8.6 Software engineering and systems engineering

Systems engineering considers complex socio-technical or physical systems such as chemical plants, factory monitoring systems, flight-control systems, buildings, and bridges. Software is often only one component of a larger system.

- A software engineer should take a broader system view when software interacts with physical equipment, people, organizations, or other subsystems.
- The engineer should understand the application/domain context sufficiently to make appropriate design decisions.
- Quality trade-offs can arise between flexibility and performance.
- Interfaces between software and non-software components can dominate system behavior and safety.

A systems perspective changes the question from 'Does the program work?' to 'Does the complete system satisfy its required behavior under its operational conditions?'

8.7 SE and other engineering disciplines

The lecture names civil engineering, mechanical engineering, and computer/hardware engineering, and raises medicine as a possible interdisciplinary connection. The broader principle is that software engineering often works inside domains with their own constraints, standards, terminology, physical processes, and safety requirements.

Discipline	Typical interaction with software engineering
Civil engineering	Infrastructure monitoring, building systems, transportation, digital twins, structural analysis.
Mechanical engineering	Control systems, robotics, CAD/CAE, manufacturing automation.
Computer/hardware engineering	Embedded systems, device drivers, processor/memory constraints, hardware–software interfaces.
Medicine	Clinical information systems, medical devices, decision-support and AI systems; strong domain, safety, privacy, and regulatory constraints.

The important exam idea is interdisciplinarity: software engineering must incorporate domain knowledge when software is part of a larger engineered product or service.

Key Points to Remember

- Know the compiler stages and the purpose of each stage.
- The compiler demonstrates separation of concerns, modularity, abstraction, and transformation.
- CS supplies theories and computational foundations; SE applies them in broader production and maintenance contexts.
- Management science addresses estimation, scheduling, people, task allocation, and tracking.
- Systems engineering requires a broader view when software is part of a complex system.
- Domain knowledge can impose constraints on software design and quality.

9. Integrated Exam Revision — High-Value Comparisons

9.1 Core definitions

Term	One-sentence exam definition
Software engineering	Disciplined, consistent, systematic construction and maintenance of software in a timely and cost-effective manner.
Software	Programs plus associated configuration data and documentation required for correct operation.
Process	A structured series of activities, constraints, and resources producing an intended output.
Fault/defect	A flaw in a software artefact resulting from a human error or other development problem.
Failure	An observed departure of system behavior from required behavior.
Correctness	Conformance/equivalence of implemented behavior to its specification.
Reliability	Probability of operating as expected over a specified time interval under stated conditions.
Robustness	Reasonable behavior under unexpected inputs or situations.
Maintainability	Ease and economy of modifying software after delivery.
Modularity	Organization of a system into focused, relatively independent parts.
Coupling	Degree of dependence/interconnection between modules.
Cohesion	Degree to which elements inside a module are strongly related to one focused purpose.
Abstraction	Representation that omits details to focus attention on other details.
Generality	Designing to avoid unnecessary context-specific restrictions so a solution can serve a broader class of cases.

9.2 Distinguish commonly confused concepts

Pair	Distinction
Correctness vs reliability	Correctness concerns conformance to specification; reliability concerns probability of correct operation over time/conditions.
Reliability vs robustness	Reliability concerns expected operation under specified conditions; robustness emphasizes unexpected inputs/situations.
Performance vs efficiency	Performance concerns work/response/throughput; efficiency concerns economical resource use.
Repairability vs evolvability	Repairability concerns fixing defects; evolvability concerns accommodating changed/new requirements.
Product vs process	Product is the delivered software/work result; process is how the software is produced.
Coupling vs cohesion	Coupling concerns relationships between modules; cohesion concerns relationships within a module.
Abstraction vs generality	Abstraction hides detail; generality broadens applicability by avoiding unnecessary specialization.
Incrementality vs waterfall	Incrementality delivers useful subsets and feedback; classical waterfall emphasizes sequential phase progression.
CS vs SE	CS emphasizes computational foundations; SE addresses systematic software production, evolution, people, process, quality, cost, and constraints.

9.3 Quality-oriented answer template

When an exam asks 'Which quality attributes are relevant?' use the following reasoning pattern:

- 1 Identify the application's domain and stakeholders.
- 2 State the required behavior and critical failure consequences.
- 3 Select relevant external qualities such as correctness, reliability, usability, security, and performance.

- 4 Select internal/evolution qualities such as verifiability, maintainability, modularity, portability, and reusability when relevant.
- 5 For each selected quality, state how it could be assessed or measured.

9.4 Principle-oriented answer template

For a design-principle question, use: **Definition** → **Purpose** → **Design consequence** → **Example** → **Trade-off**. For example, for low coupling: define it as weak inter-module dependence; explain that it reduces change propagation; give a data-passing interface as an example; then mention that interfaces still create necessary dependencies and must be designed carefully.

10. Rapid Revision Sheet

10.1 Lifecycle and process

Lifecycle: Requirements → Architecture/System Design → Detailed Design → Coding → Unit/Integration/System Testing → Deployment → Maintenance.

Framework activities: Communication → Planning → Modeling → Construction → Deployment.

Umbrella activities: SQA, Configuration Management, Technical Reviews, Project Tracking & Control, Risk Management.

10.2 Quality map

- Correctness → conforms to specification.
- Reliability → probability of expected operation over time.
- Robustness → reasonable behavior under unexpected conditions.
- Performance → work/response/throughput.
- Efficiency → economical resource usage.
- Usability → human ease and effectiveness of use.
- Verifiability → ease of checking properties.
- Maintainability → ease/cost of modification.
- Repairability → ease/cost of defect correction.
- Evolvability → ease/economy of satisfying changed requirements.
- Reusability → ability to use artefacts in other contexts.
- Portability → ability to run in other environments.
- Understandability → ease of comprehension.
- Interoperability → ability to cooperate with other systems.
- Security → protection of information and resistance to unauthorized access/misuse.

10.3 Principles map

- **Rigor/Formality:** precision, exactness, mathematical/formal reasoning where appropriate.
- **Separation of concerns:** isolate unrelated dimensions.
- **Modularity:** focused independent parts; decomposability + composability.
- **Low coupling:** minimize unnecessary dependencies between modules.
- **High cohesion:** keep closely related responsibilities together.
- **Anticipation of change:** design likely variation points deliberately.
- **OCP:** open for extension, closed for modification.
- **Generality:** avoid unnecessary context-specific assumptions.
- **Incrementality:** deliver early subsets and obtain feedback.
- **Abstraction:** hide detail to focus reasoning.
- **Documentation/traceability:** preserve assumptions and connect artefacts.
- **Reuse:** know existing assets and design clear reusable interfaces.

10.4 Compiler case in one line

Characters → Tokens → Syntax tree → Semantic checks → Intermediate representation → Optimization → Target code.

10.5 Final exam checklist

- Can I define software engineering in one precise sentence?
- Can I distinguish software from source code alone?
- Can I explain the roles of requirements analyst, designer, programmer, tester, trainer, maintenance, documentation, and configuration management?
- Can I write the lifecycle phases in order?
- Can I distinguish error, fault/defect, and failure?
- Can I explain why software evolves and state the four Lehman laws taught here?
- Can I define and compare the major quality attributes?
- Can I explain McCall, FURPS, and ISO/IEC 25010 at the level taught in the slides?
- Can I classify software into the major application domains?
- Can I explain legacy software characteristics and why it is risky to evolve?
- Can I explain the five framework activities and five umbrella activities?
- Can I define rigor, formality, separation of concerns, modularity, coupling, and cohesion?
- Can I identify coupling and cohesion types using examples?
- Can I explain over- versus under-modularization?
- Can I define OCP, generality, incrementality, and abstraction and state their trade-offs?
- Can I explain the 'customer of another phase' idea and traceability?
- Can I explain the compiler pipeline and relate it to SE principles?
- Can I distinguish software engineering from computer science, management science, and systems engineering?

End of notes

These notes are designed to function as a consolidated revision source. For any question whose wording depends on an exact classroom diagram, numerical statistic, or slide-specific illustration, reproduce the terminology and framing used by the instructor and use the conceptual explanations here to support it.